

МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОННОЙ ТЕХНИКИ

Институт системной и программной инженерии
и информационных технологий (Институт СПИНТех)

Лабораторная работа № 1

Трудоёмкость алгоритма обработки данных.

Моделирование функций активации нейрона

Выполнил:

Трусов М.П. гр. ПИН-42

Проверил преподаватель:

проф., д.ф.-м. н. Рычагов М.Н.

Москва, 2026

3.1. Реализация ДПФ.

Пусть задан гармонический сигнал с некоторыми амплитудой и частотой. Изучить программу *Lab_1_1.ipynb*, обеспечивающую диалоговое задание гармонического сигнала и его визуализацию, а также реализующую ДПФ такого сигнала и его восстановление с помощью обратного ДПФ. Пояснить работу программы, выбор частоты дискретизации и исчезновение оператора суммы при программной реализации прямого и обратного ДПФ, которое имеет место в формулах (16) и (17).

Введённые параметры:

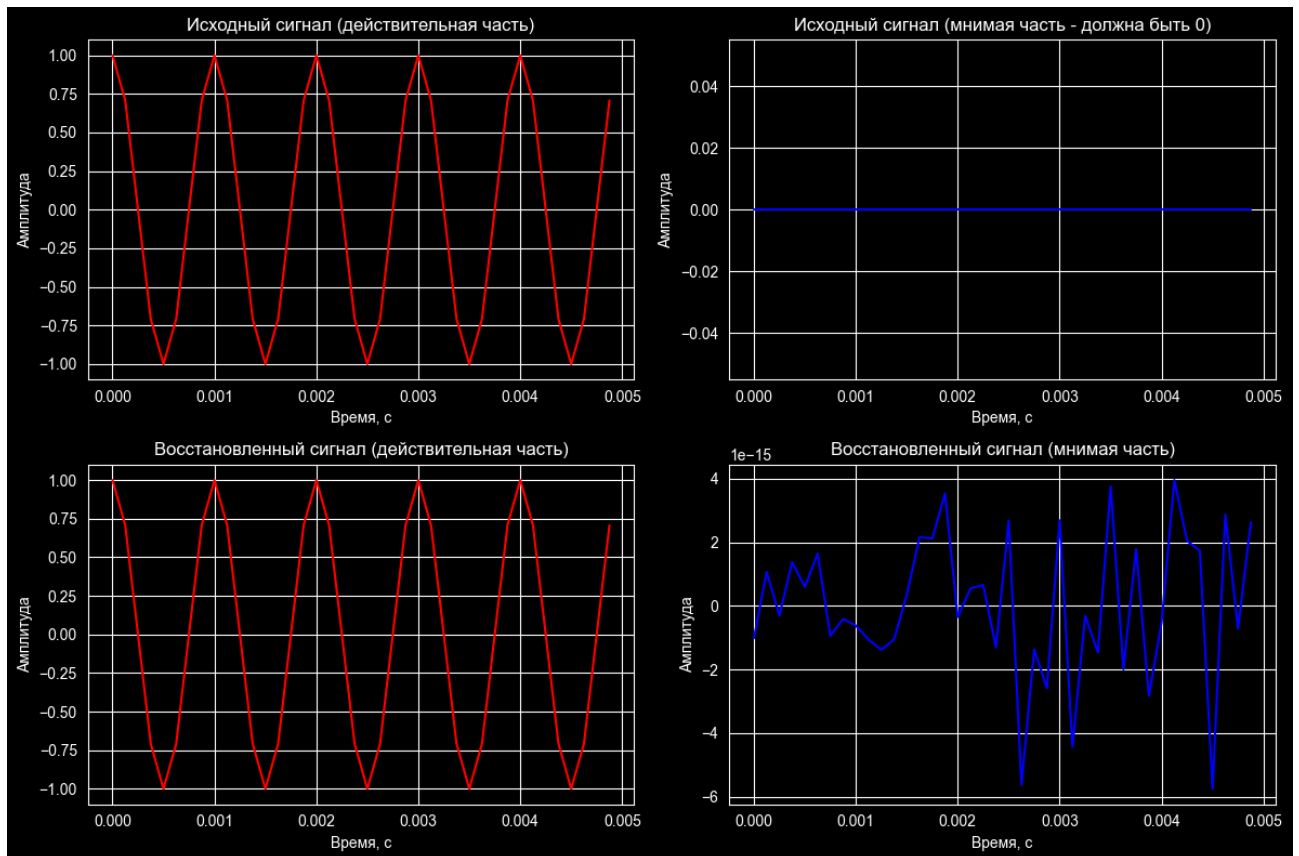
Амплитуда сигнала – 1 ед.;

Частота сигнала – 1 кГц.;

Фаза сигнала – 0 рад.;

Количество периодов наблюдения – 5;

Коэффициент увеличения дискретизации – 4.



Выбор частоты дискретизации основывается на теореме Найквиста, которая гласит, что для восстановления сигнала частота дискретизации должна быть как минимум вдвое выше максимальной частоты в сигнале.

Исчезновение оператора суммы происходит благодаря тому, что операция матричного умножения выполняет это суммирование автоматически.

3.2. Оценка трудоемкости обработки данных с помощью ДПФ и БПФ

Используя программу *Lab_1_1.ipynb*, написать программу *Lab_1_2.ipynb*, реализующую:

а) дискретизацию и визуализацию функций синуса и косинуса с частотой 2 кГц в двух вариантах: для заданного интервала наблюдения и для заданного количества точек;

Введённые параметры:

Амплитуда сигнала – 1 ед.;

Фаза сигнала – 0 рад.;

Интервал наблюдения – 0.001 с.;

Количество точек: 50.

Фрагмент программного кода:

```
A = float(input('Введите амплитуду сигнала, ед.: '))
f0 = 2000 # Частота сигнала, Гц
phi = float(input('Введите фазу сигнала, рад: '))

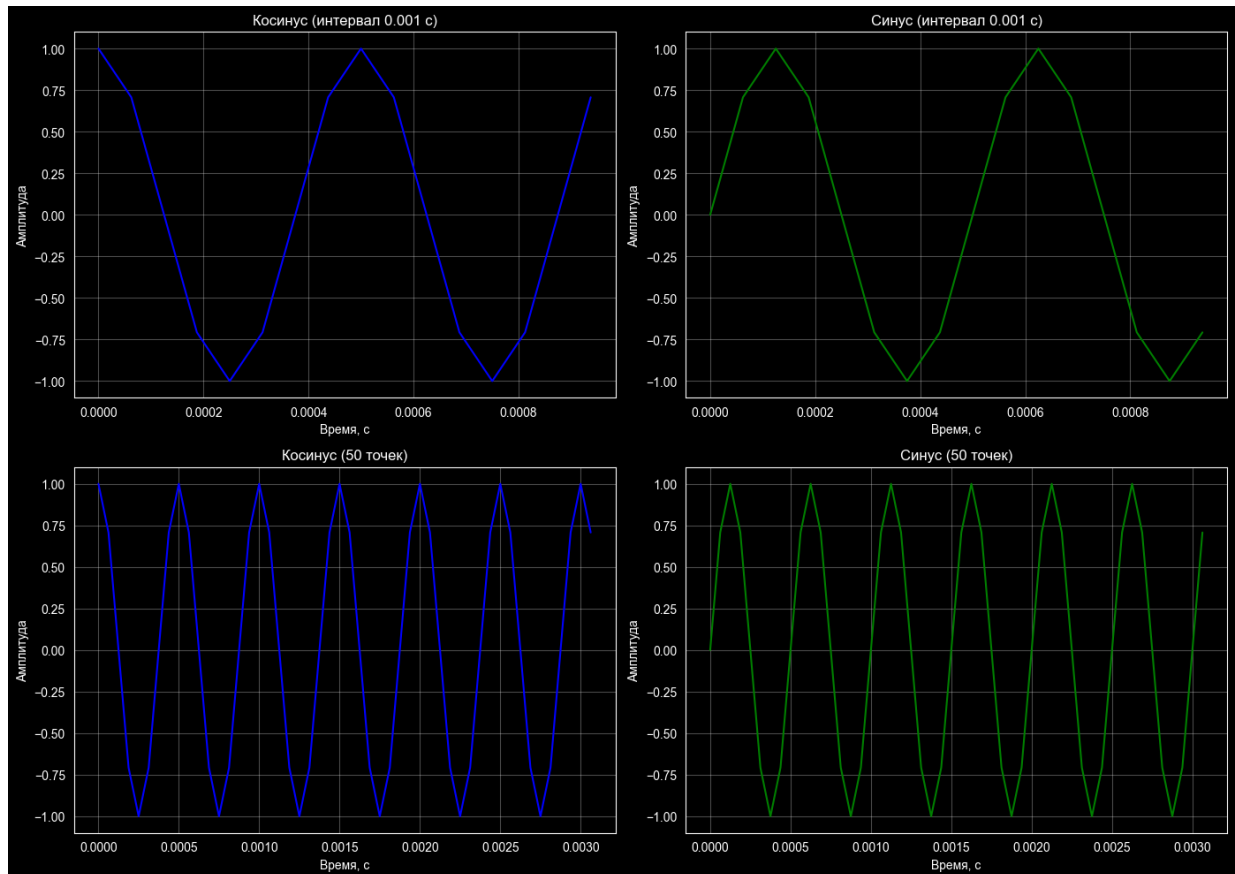
T = 1 / f0 # Период сигнала
fdn = 2 * f0 # Частота Найквиста-Котельникова
mvis = 4 # Коэффициент увеличения дискретизации
fdv = mvis * fdn # Частота дискретизации для визуализации
dt = 1 / fdv # Интервал дискретизации

T_nab = float(input('Введите интервал наблюдения (с): '))
t1 = np.arange(0, T_nab, dt) # Вектор времени

# Сигнал
y_cos1 = A * np.cos(2 * np.pi * f0 * t1 + phi)
y_sin1 = A * np.sin(2 * np.pi * f0 * t1 + phi)
#%%
N_points = int(input('Введите количество точек: '))
t2 = np.arange(0, N_points * dt, dt)

# Сигнал
y_cos2 = A * np.cos(2 * np.pi * f0 * t2 + phi)
y_sin2 = A * np.sin(2 * np.pi * f0 * t2 + phi)
```

Вывод графика:



б) вычислить фурьеобразы исходных сигналов с помощью прямого вычисления ДПФ и с помощью ДПФ, реализованного в Python (функция `np.fft.fft()`);

Программный код:

```
def dpf_direct(y):  
    N = len(y)  
    k = np.arange(N)  
    Ex = np.exp(-1j * 2 * np.pi / N * np.outer(k, k))  
    Y = y @ Ex  
    return Y
```

```
t1 = np.arange(0, T_nab, dt)
```

```
cos_orig = A * np.cos(2 * np.pi * f0 * t1 + phi)  
sin_orig = A * np.sin(2 * np.pi * f0 * t1 + phi)
```

```
Y_cos_direct = dpf_direct(cos_orig)  
Y_cos_fft = np.fft.fft(cos_orig)
```

```
Y_sin_direct = dpf_direct(sin_orig)  
Y_sin_fft = np.fft.fft(sin_orig)
```

```
print(f"Максимальное различие для косинуса:  
{np.max(np.abs(Y_cos_direct - Y_cos_fft)):.2e}")
```

```
print(f"Максимальное различие для синуса:
{np.max(np.abs(Y_sin_direct - Y_sin_fft)):.2e}")
```

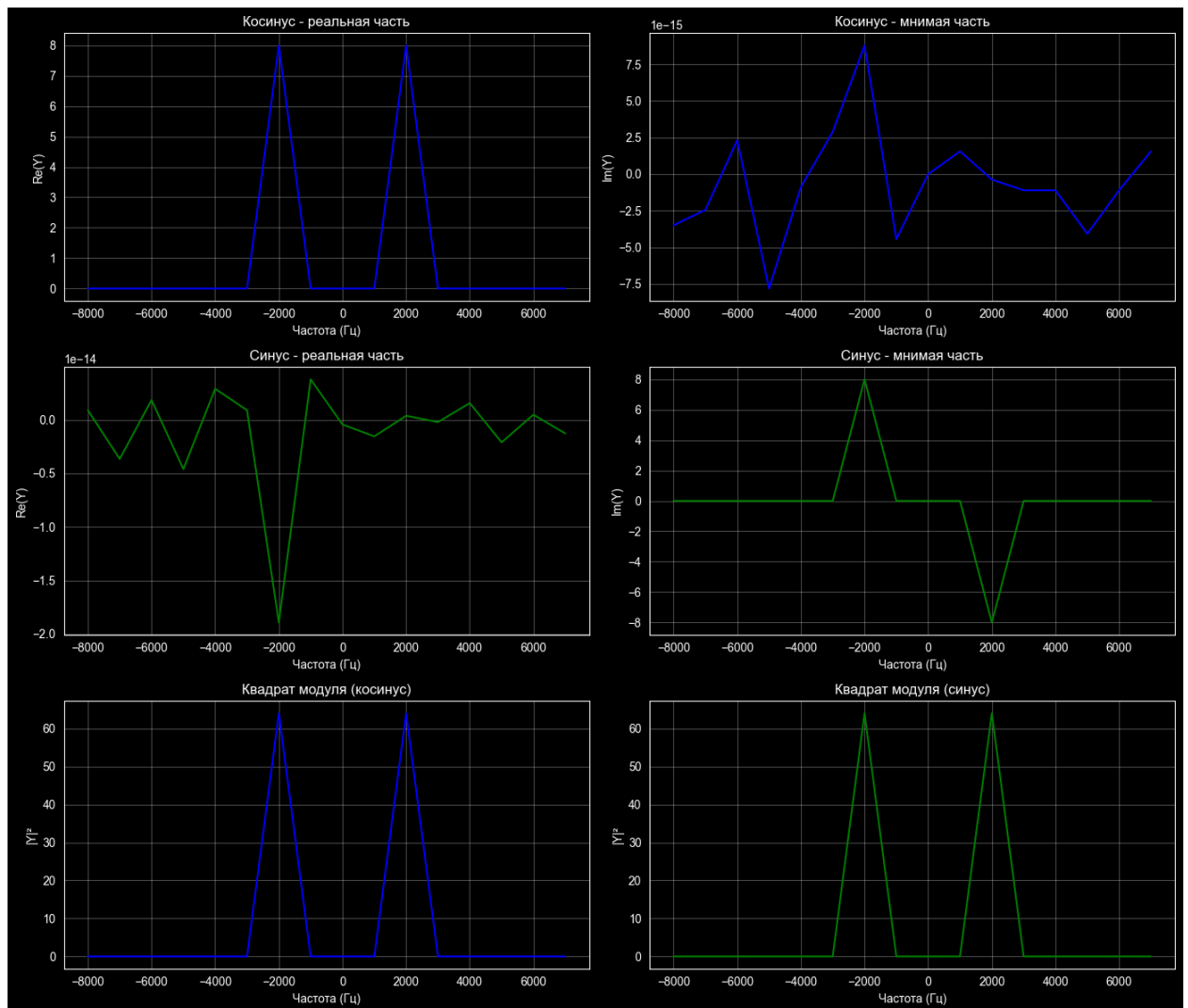
Вывод программы:

Максимальное различие для косинуса: 1.12e-14

Максимальное различие для синуса: 1.73e-14

в) визуально сравнить реальные и мнимые части фурье-образов и квадраты их модулей.

Вывод графика:



Построить график зависимости времени обработки исходных данных с помощью ДПФ и БПФ, варьируя размерность исходного массива 2 от 128 ($s = 7$) до 4096 ($s = 12$) (если не происходит зависание вычислительного устройства).

Фрагмент программного кода:

```
N_values = [2 ** s for s in range(7, 14)]
```

```
time_dft, time_ffft = [], []
```

```

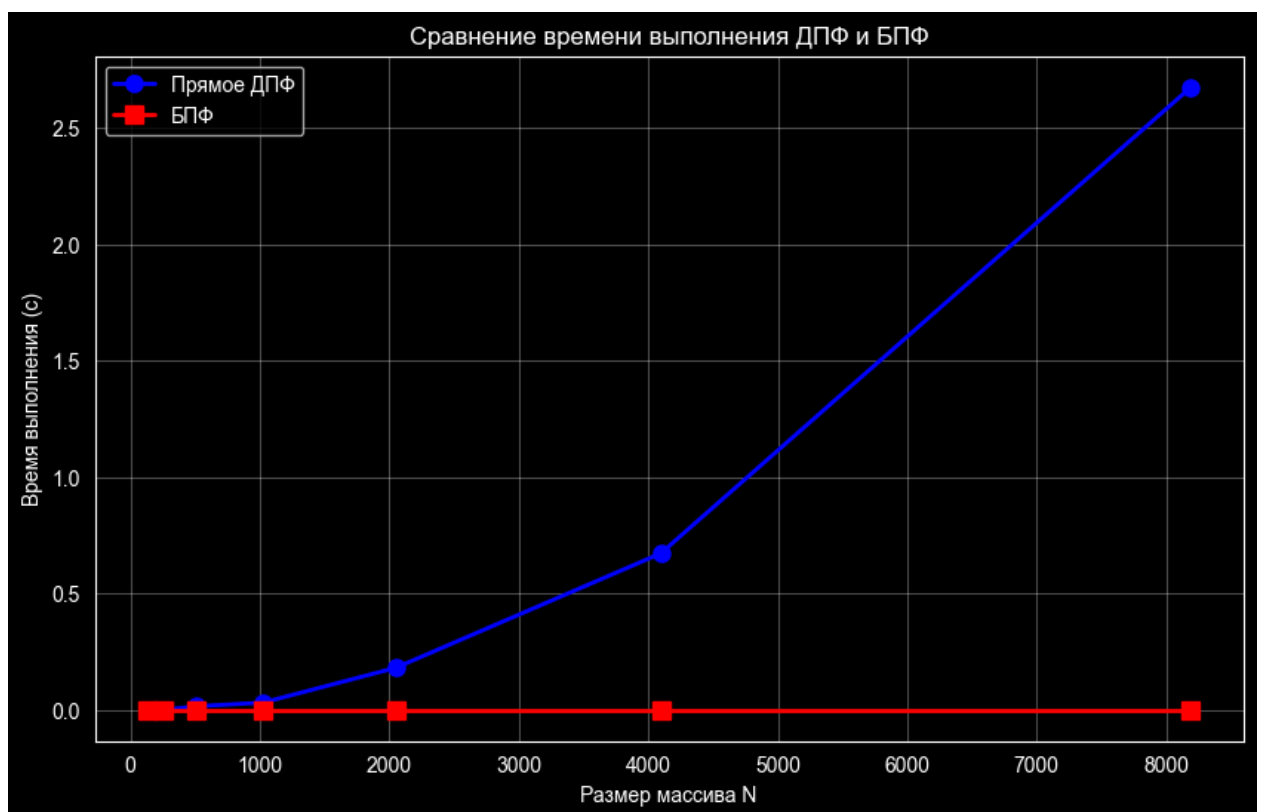
for N in N_values:
    t = np.arange(0, N * dt, dt)
    signal = A * np.cos(2 * np.pi * f0 * t + phi)

    # ДПФ
    start = time.time()
    Y_dft = dpf_direct(signal)
    end = time.time()
    time_dft.append(end - start)

    # БПФ
    start = time.time()
    Y_fft = np.fft.fft(signal)
    end = time.time()
    time_fft.append(end - start)

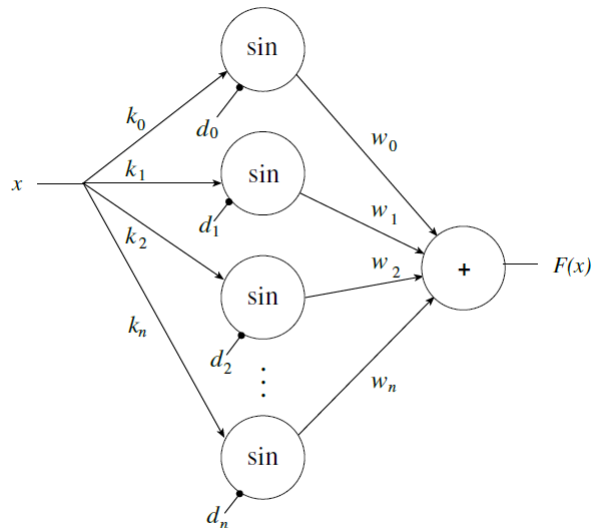
```

Вывод графика:



3.3. Архитектура нейронной сети

На рисунке представлена архитектура нейронной сети, реализующая некоторую функциональную зависимость. Если принять, что $d_0, \dots, d_n$ – фазовые сдвиги, задаваемые в виде $d_0, \dots, d_n$, $d_0 = \pi/2$, а примитивы (простые функции) заданы в виде $\sin(x)$, то что реализует данная архитектура?



$$F(x) = k_0 * \cos(x) + k_1 * \sin(x + d_1) + k_2 * \sin(x + d_2) + \dots + k_n * \sin(x + d_n)$$

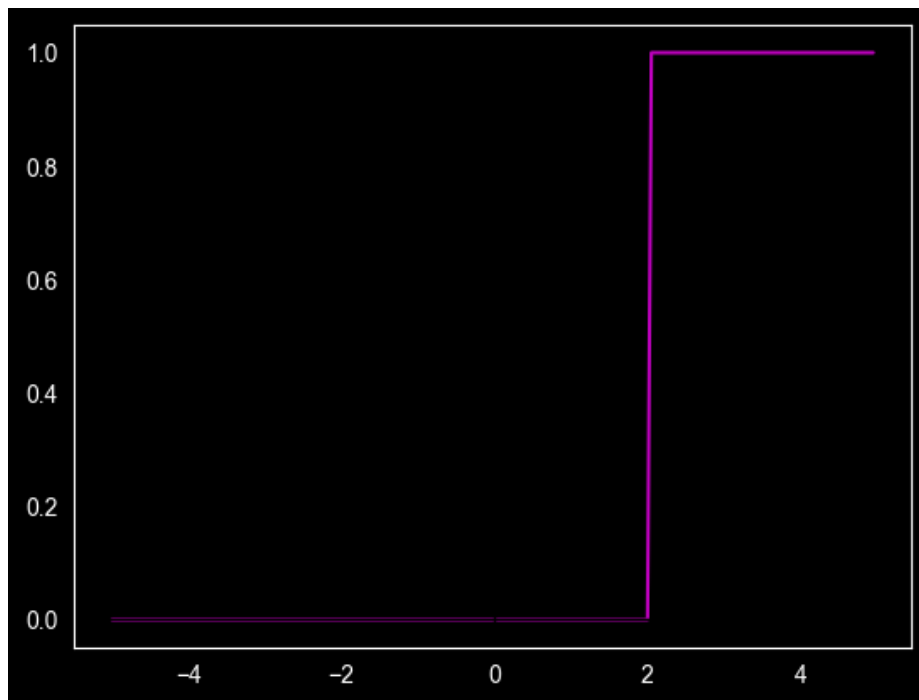
Нейронная сеть образует ряд Фурье.

3.4. Программирование функций активации нейрона (перцептрона)

Написать программу-функцию, реализующую вычисление и отображение функций активации, представленных в разделе 2. Результат представить в виде функции, на вход которой поступает массив входных данных v , а также, если требуется, параметр a , а в результате ее выполнения производится прорисовка требуемой функции активации.

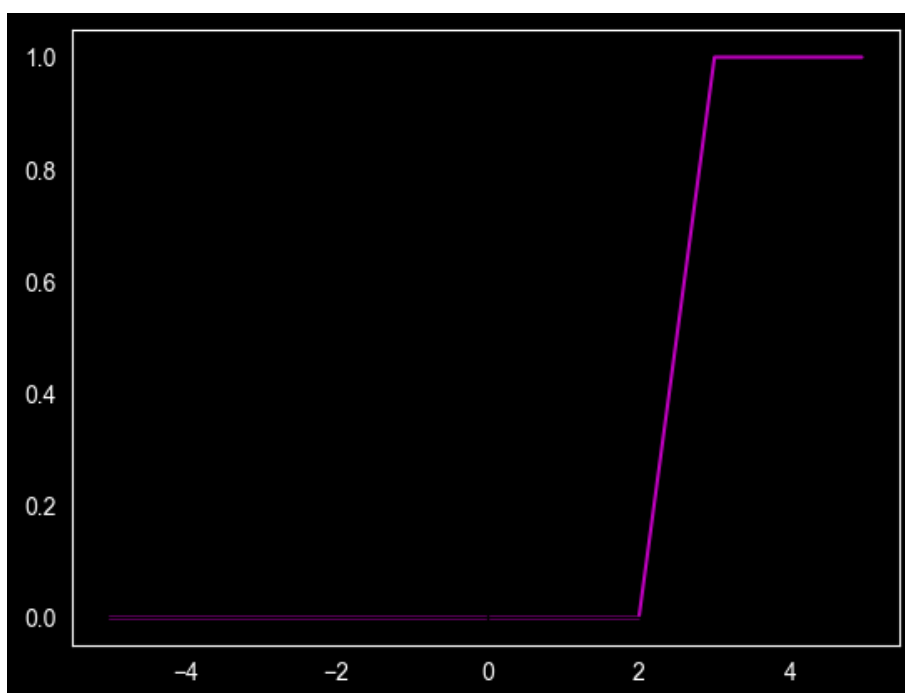
Единичный скачок или пороговая функция, т.е. простая кусочно-линейная функция.

```
def threshold_function(x, v0):
    plt.plot(x, (x > v0).astype(int), 'm')
    plt.axhline(0, c='k', lw=1)
    plt.axvline(0, c='k', lw=1)
    plt.grid()
    plt.show()
threshold_function(x, 2);
```



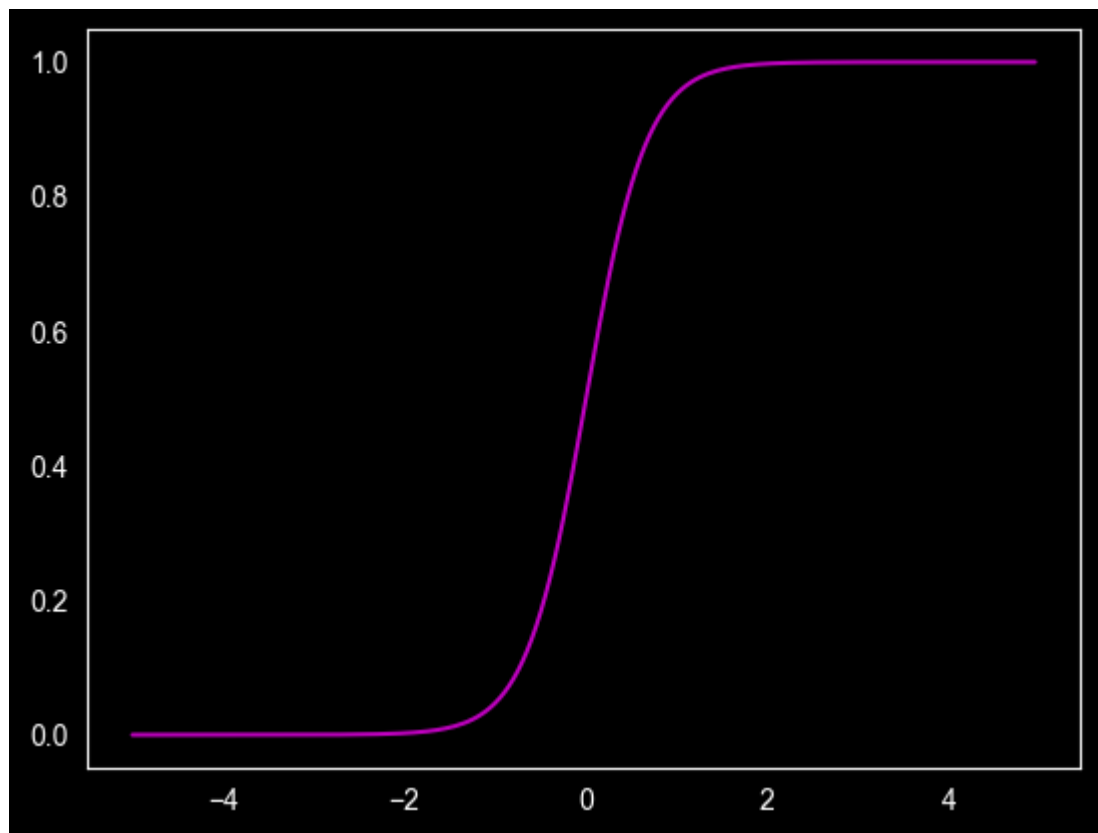
Линейный порог или гистерезис, т.е. кусочно-линейная функция.

```
def hysteresis(x, v0, v1):
    plt.plot(x, np.clip((x - v1) / (v0 - v1), 0, 1), 'm')
    plt.axhline(0, c='k', lw=1)
    plt.axvline(0, c='k', lw=1)
    plt.grid()
    plt.show()
hysteresis(x, 3, 2);
```



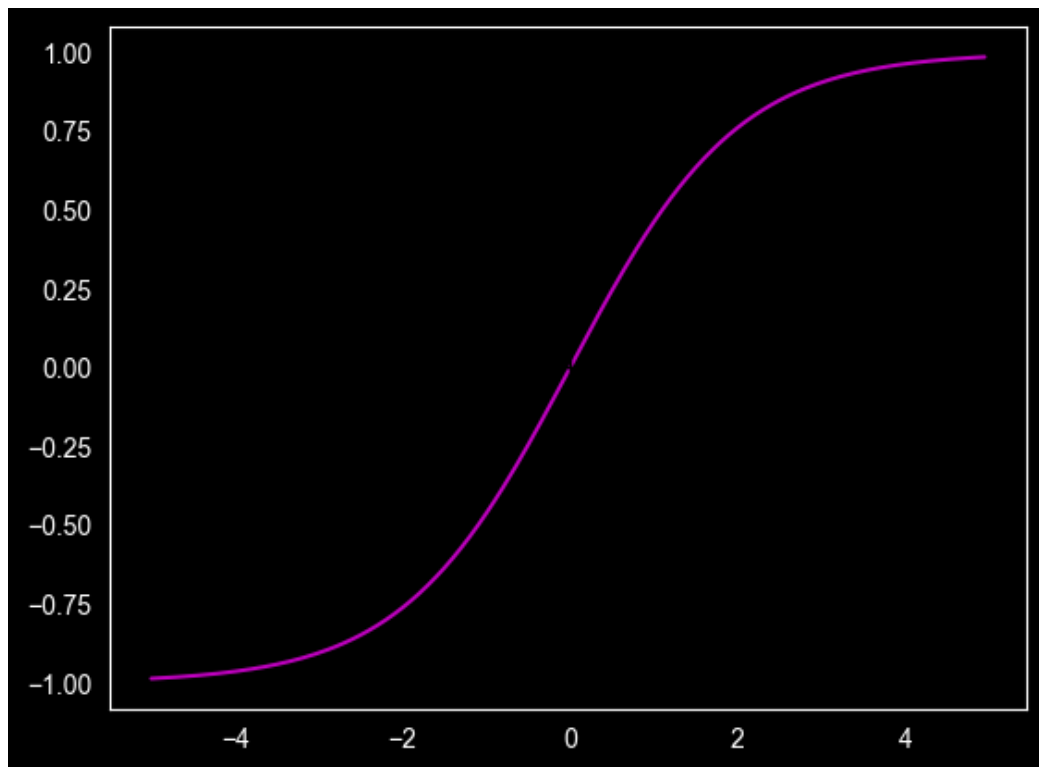
Сигмоидная функция, а именно монотонно возрастающая, всюду дифференцируемая S-образная нелинейная функция с насыщением.

```
def sigmoid(v, alpha):  
    plt.axhline(0, c='k', lw=1)  
    plt.axvline(0, c='k', lw=1)  
    plt.plot(v, 1 / (1 + np.exp(-alpha * v)), 'm')  
    plt.grid()  
    plt.show()  
sigmoid(x, 3);
```



Еще одним примером сигмоидной функции активации является гиперболический тангенс.

```
def hyperbolic_tangent(v, alpha):  
    plt.plot(v, np.tanh(v / alpha), color='m')  
    plt.axhline(0, color='black', linewidth=1)  
    plt.axvline(0, color='black', linewidth=1)  
    plt.grid()  
    plt.show()  
hyperbolic_tangent(x, 2);
```



3.5. Производная сигмоидной функции

Вычислите (теоретически и численно) производную сигмоидной функции (п. 2.3) и представьте на графике.

Теоретическая производная сигмоиды: $\alpha * y * (1 - y)$

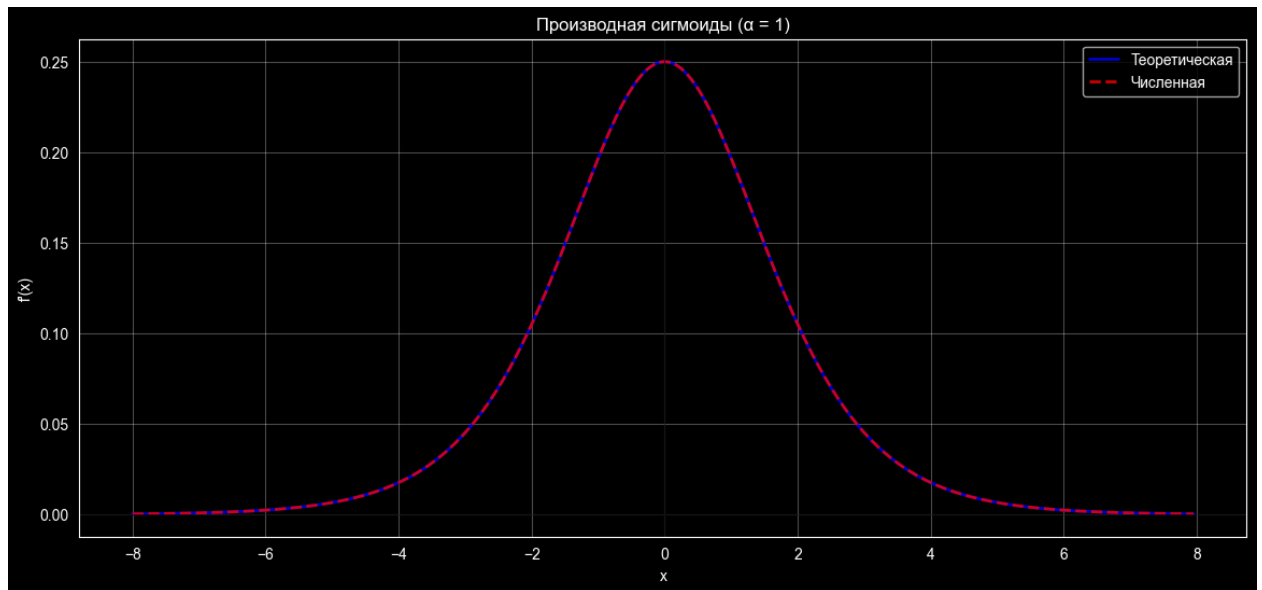
Для численной производной воспользуемся формулой центральной разности.

Фрагмент программного кода:

```
def sigmoid(v, alpha):  
    return 1 / (1 + np.exp(-alpha * v))  
def sigmoid_derivative_theoretical(v, alpha):  
    y = sigmoid(v, alpha)  
    return alpha * y * (1 - y)  
def sigmoid_derivative_numerical(v, alpha, eps=1e-2):  
    return (sigmoid(v + eps, alpha) - sigmoid(v - eps, alpha)) /  
    (2 * eps)  
  
alpha = 1  
v = np.arange(-8, 8, 0.05)
```

```
y_theoretical = sigmoid_derivative_theoretical(v, alpha)
y_num = sigmoid_derivative_numerical(v, alpha)
```

Вывод графика:



Максимальная разница между теоретической и численной производной: 2.08×10^{-6}